SECCIÓN 3: PREGUNTAS TEÓRICAS

Pregunta 1: Arquitectura de Backend El documento sugiere usar Node.js con los frameworks Express o NestJS. ¿Cuáles son las principales diferencias arquitectónicas entre Express y NestJS, y en qué tipo de proyectos o escenarios recomendarías utilizar uno sobre el otro?

Respuesta: Una de las diferencias entre estas dos tecnologías es principalmente su manera de diseño, Express.js usa por defecto JavaScript, mientras que Nest.js corre bajo el uso de TypeScript

Usar Express.js es ideal para el desarrollo mas simple, en el uso de APIs a corto tiempo ya que suele ser más rápido de usar y tiene una curva de aprendizaje baja.

Mientras que Nest.js es mas recomendado para aplicaciones de escala empresarial que buscan una mantenibilidad a largo plazo y un trabajo colaborativo.

Pregunta 2: Bases de Datos (SQL vs NoSQL) Para la persistencia de datos, la prueba da la opción de elegir entre MongoDB (usando Mongoose) o PostgreSQL (usando TypeORM). Explica las diferencias teóricas clave entre un motor de base de datos relacional y uno no relacional. ¿Qué factores tomarías en cuenta para elegir la base de datos ideal para una aplicación de tareas a gran escala?

Respuesta: La diferencia entre una y otra es la manera en la que se modelan, almacenan y garantizan la consistencia de los datos o la información.

Por ejemplo, las relacionales se rigen bajo tablas esto define relaciones explicitas, el uso de claves primarias y foráneas.

Mientras que las no relacionales se basan mediante documentos, JSON estas son mas flexibles y estas se agrupan en colecciones.

Así como la rapidez de las consultas entre ambas es mas notoria en las No Relacionales.

Mi recomendación para la mayoría de sistemas de tareas a gran escala las Relacionales suelen ser la mejor opción, ya que estas igual cuentan con mayor seguridad además estas dependen fuertemente de la integridad de las relaciones y del estricto control transaccional.

Pregunta 3: Seguridad y Middlewares Se solicita proteger las rutas de la API mediante un middleware de autenticación. Describe teóricamente cómo funciona el ciclo de vida de una petición HTTP que pasa por un middleware de autenticación. Además, explica brevemente el flujo de trabajo de la autenticación basada en JWT (JSON Web Tokens) y por qué es preferida en APIs RESTful.

Respuesta: El ciclo de vida de una petición HTTP comienza en la recepción del Request, donde un cliente envia una petición HTTP a una ruta (protegida).

Seguido de la intercepción, aquí es donde la petición entra a un pepeline del servidor y es interceptada por el middleware de autenticación.

extracción y la validación: extracción del token del encabezado, si no hay token, se interrumpe el flujo y retorna una respuesta de error.

Toma de decisión: si el token es valido extrae la información y la adjunta al objeto de la petición.

Respuesta: el controlador procesa la solicitud usando los datos del request.user y devuelve la respuesta HTTP al cliente.

Flujo de trabajo de la autenticación basada en JWT:

1.- La Autenticación: el cliente envía sus credenciales al endpoint de autenticación.

2.- Generacion del token: el servidor valida las credenciales contra la base de datos, si son correctas, este genera un JWT compuesto por 3 partes separadas por puntos Header.Payload.Signature.

3.- Almacenamiento en cliente ya sea LocalStorage, SessionStorage o cookies.

4.- el cliente adjunta el JWT en el encabezado Authorization de cada solicitud

5.- Peticiones: el servidor valida el token usando su clave secreta, si estos coinciden el servidor confía en los datos.

¿Por qué usar JWT en APIS RESTful?

Se prefiere el uso de JWT ya que este permite una arquitectura completamente sin estado al empaquetar la información del usuario directo al token. Esto facilita la

escalabilidad, ya que el servidor puede validarlo de forma independiente usando la misma clave secreta.

Pregunta 4: Gestión del Estado Global en Frontend En el desarrollo del frontend con Vue (Nuxt 3), se requiere explícitamente el uso de Pinia para manejar el estado global de la aplicación. ¿Qué problemas arquitectónicos resuelve un gestor de estado global como Pinia en comparación con pasar props y emitir eventos de forma tradicional entre múltiples componentes anidados?

Respuesta: resuelve principalmente el problema del "Prop Drilling" y el acoplamiento excesivo, eliminando la necesidad de pasar datos a través de múltiples capas de componentes intermedios que no los necesitan solo para hacerlos llegar a un componente hijo profundo. Además, centraliza y desacopla la lógica de negocio, evitando que el estado quede disperso en componentes individuales o se desincronice en aplicaciones complejas.

Pregunta 5: Estrategias de Renderizado El ejercicio de la interfaz de usuario específica que el candidato puede crear una SPA (Single Page Application) o una SSR (Server-Side Rendering). Define teóricamente ambos conceptos e indica cuáles son las ventajas y desventajas de utilizar SSR con Nuxt 3, haciendo especial énfasis en el impacto sobre el SEO y el rendimiento (First Contentful Paint)

Respuesta: Una SPA es un a arquitectura donde el cliente descarga un HTML prácticamente vacío junto con el ejecutable de JavaScript y renderiza la interfaz en el navegador, mientras que el SSR procesa las peticiones en un servidor Node.js que consulta los datos y entrega al cliente el marcado HTML completamente cosntruido y poblado. Hacer un SSR con NUXT 3 da ventajas que ayudan a optimizar drásticamente el SEO por que los motores de búsqueda e indexadores pueden analizar de inmediato el texto.

Sus desventajas es que incluyen un mayor costo y carga de cómputo en la infraestructura del servidor, un posible aumento en el Time to First Byte si las peticiones a la API son lentas.